Discrete and Continuous Models for Concurrent Systems

2. Concurrent semantics of Petri nets

Uli Fahrenberg
LMF, Université Paris-Saclay, France

ISIMM, April 2026

Exercise 0 You will need to two tools for these exercises:

- A graphical editor for Petri Nets that also lets us "execute" them on the fly.
  We will use `wolfgang` for this:
  https://github.com/iig-uni-freiburg/WOLFGANG
  We recommend using the jar.
  It can be run by simply using:
  `java -jar wolfgang-1.0.3.jar`
  When launched, chose place/transition net.
  Once your Petri Net construct with all its places, transitions and initial marking, you can simulate an execution by clicking `edit`.
  Fireable transition are marked red and are executed by clicking them. This will consume and produce the corresponding tokens.

- `pn2HDA`: A tool for translating Petri Nets into HDAs and perform several actions on them. You can either:
  - Use the provided docker
  - Install it on your machine

Setting up the provided docker

First you need to pull the docker image:
`docker pull elfuius/pn2hda`
Once this is completed, go to your working directory (where you have your pnml files and where you want to produce the output) and use
`docker run --it -rm -v $(pwd):/home/work elfuius/pn2hda`
This will run docker image, bind the current folder under `/home/work` and give you access to a console within the docker.
Now switch to your working folder inside the docker image:
`cd /home/work`.
It should have the same content as your current folder. Please go to **Using pn2HDA** now.

Installing `pn2HDA` locally

First you need to install `Symmetri`:

Clone the repo
`git clone https://github.com/thorstink/Symmetri.git`
Setup and install

```
git submodule update --init --recursive\\
mkdir build\\
cd build\\
# Release build
cmake -DCMAKE_BUILD_TYPE=Release ..
# Building and installing:
make && sudo make install
```

Once this is done you can install `pn2HDA`:

```
git clone https://gitlabev.imtbs-tsp.eu/philipp.schlehuber-caissier/pn2hda.git \\
cd pn2hda \\
cmake -DCMAKE_BUILD_TYPE=Release -S . -B build \\
cmake --build build \\
```

Now you should be all setup. To display the ST-automaton corresponding to an HDA as pdf you will also need `dot`, usually provided by the `graphviz` package.
If you use a local installation, `pn2HDA` below must be replaced by `/your/path/to/build/pn2HDA`.

Using pn2HDA
Given that there exists a `test.pnml` petri in your current folder you can:

- A pdf showing the corresponding ST-automaton like so:
  `pn2HDA standard test.pnml dot > test.dot && dot -Tpdf -o test.pdf test.dot`
- Show a summary of statistics of the HDA like so:
  `pn2HDA standard test.pnml "csv_header"`

Exercise 1 Implement the "Bakery" Petri net from the lecture in `wolfgang` and simulate its steps. Save it as `bakery.pnml` and have a look at that file in a text editor.

Run the net until you have explored all reachable markings. Draw the reachability graph of the net and its HDA semantics. Use `pn2hda` to confirm your results.

Exercise 2 Implement the last example of the lecture in `wolfgang` and translate it to an ST-automaton. Add a new place and a transition d which *disables* the concurrency of a and b. Repeat the translation to an ST-automaton. What does the corresponding HDA look like?

A good engineer knows how to reverse engineer

Till here we have given you the Petri Nets to analyze, now we ask to go into the other direction! We give you the cell statistics, that is the number of cells per dimension, of the HDA and you need to find a Petri Net together with its initial marking producing the same statistics.

Note that in general this is not without ambiguity, that is there can be distinct nets having the same cell statistics, however for small HDAs the choice is fairly limited. There are probably more exercises than there is time, don't stress it is more beneficial to properly understant it. If you discover during this winter school that this is something that really interests you → We will probably have open PhD positions next fall, don't hesitate to contact us!

Exercise 3 Warm-Up!
Which HDA has

dim	#
0	4
1	4
2	1

?

Can you construct the corresponding Petri Net?

Exercise 4 Let's talk equivalence
Remember that each cell in the HDA corresponds to a distinct tuple $(marking, conclist)$, which we also call configuration.

If the same configuration of the Petri Net can be attained using *different* paths, they end up in the same cell. This can cause a cell to have "fewer" sub-cells as expected, since some of them are identical.

This can be used to generate HDAs like this one:

dim	#
0	3
1	1
2	1

.

What does the geometry look like? Which Petri Net generates such an HDA?

Exercise 5 A parametric case
We will now consider an HDA parametrized by its number of 2-cells called N. For a given N we get the corresponding cell statistics

dim	#
0	2(N+1)
1	3N+1
2	N

.

Exercise 6 Mhh tasty

Can you somehow modify the above example to achieve the following distribution

dim	#
0	N
1	2N
2	N

.

Hint: Note that all examples had deadlocks (a marking from which not transition can be fired), in fact every execution eventually ended in a deadlock. This is not the case for this example.

Exercise 7 2 dimensional does not really put the HIGH in HDA does it?

Let's go one step further and construct something cube-like:

dim	#
0	20
1	30
2	24
3	10

!